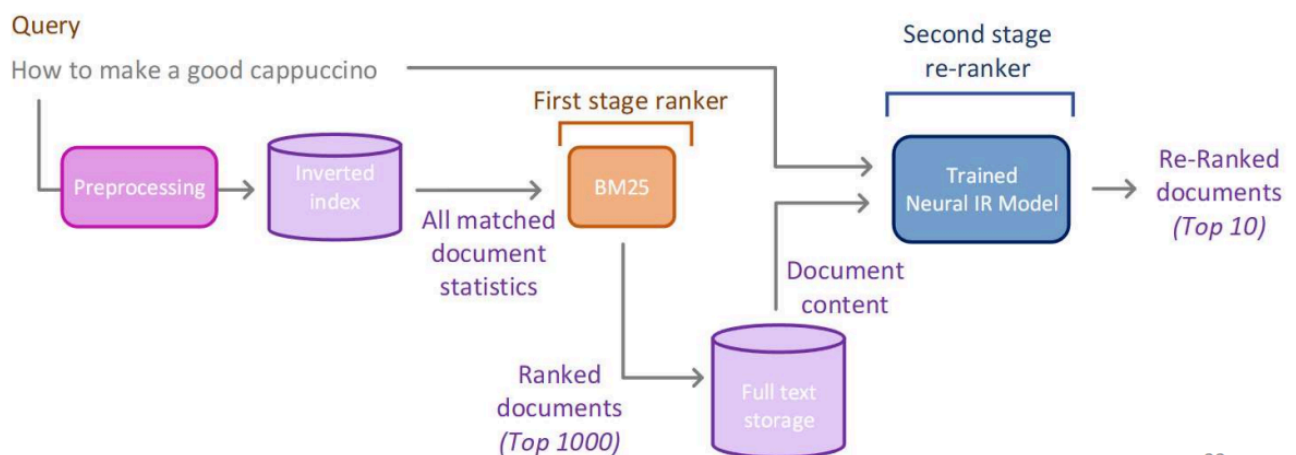


Τρίτη Φάση Προγραμματιστικής Εργασίας στην Σημασιολογική Ανάλυση

Υλοποίηση Υβριδικού Συστήματος Ανάκτησης Πληροφοριών με Χρήση BM25 και
Σημασιολογικής Επαναταξινόμησης (Semantic Re-ranking)

Η τρίτη φάση της εργασίας υλοποιεί ένα υβριδικό σύστημα σημασιολογικής αναζήτησης που συνδυάζει την κλασική λεκτική αναζήτηση με τη σημασιολογική ανάλυση. Η Elasticsearch (BM25) λειτουργεί ως φίλτρο για να περιορίσει το χώρο αναζήτησης στα 200 πιο σχετικά, προσφέροντας ταχύτητα, η οποία μας έλλειπε στην 2η φάση. Ο Transformer (all-MiniLM-L6-v2) αναλαμβάνει να επαναταξινομήσει αυτά τα 200 έγγραφα με βάση τη σημασιολογική τους ομοιότητα. Τέλος, όπως και στην πρώτη φάση, ασχοληθήκαμε και με το εργαλείο αξιολόγησης *trec_eval*, το οποίο μας έδωσε *map* και *precision* για τις απαντήσεις μας και έτσι μπορέσαμε να τις αξιολογήσουμε. Συγκεκριμένα βήμα-βήμα:

1. Χρησιμοποιήσαμε την Elasticsearch για την ανάκτηση των $N=200$ επικρατέστερων εγγράφων βάσει BM25(κρατήσαμε το ενδεικτικό N μιας και φαινόταν να δίνει καλύτερα αποτελέσματα), αξιοποιώντας τον κώδικα της 1ης φάσης.
2. Χρησιμοποιήσαμε το all-MiniLM-L6-v2 για τη δημιουργία διανυσμάτων (embeddings) μόνο για τους 200 υποψηφίους, αξιοποιώντας τον κώδικα της 2η φάσης.
3. Χρησιμοποιήσαμε την βιβλιοθήκη FAISS για τον υπολογισμό του Cosine Similarity (μέσω IndexFlatIP σε normalized vectors) μεταξύ του ερωτήματος και των 200 εγγράφων.



27

Επιλέξαμε να σταθμίσουμε με Linear Score Fusion. Αρχικά, χρησιμοποιήσαμε Min-Max Normalization στο BM25 για να φέρουμε τα λεκτικά και σημασιολογικά σκορ σε κοινή κλίμακα, επιτρέποντας τη σωστή λειτουργία της υβριδικής στάθμισης([0,1]). Ύστερα, επιλέξαμε βάρος $w=0.7$ για το BM25 διότι όπως φάνηκε και στη 2η φάση της εργασίας, έδινε καλύτερα αποτελέσματα.

```

94     bm25_scores = [hit["_score"] for hit in hits]
95
96     # Normalizing BM25 scores to [0, 1] range for better combination with cosine similarity scores
97     if len(bm25_scores) > 1:
98         max_bm = max(bm25_scores)
99         min_bm = min(bm25_scores)
100
101         # normalize: (score - min) / (max - min)
102         norm_bm25 = [(s - min_bm) / (max_bm - min_bm + 1e-9) for s in bm25_scores]
103     else:
104         norm_bm25 = [1.0] * len(bm25_scores)
105
106     candidate_texts = [hit["_source"]["Text"] for hit in hits]
107     candidate_ids = [hit["_id"] for hit in hits]
108
109     # Copying the code from phase2.py to compute semantic similarity scores for the candidate documents and combine them with BM25 scores
110     embeddings = model.encode(
111         candidate_texts,
112         batch_size=64,
113         show_progress_bar=True
114     )
115     faiss.normalize_L2(embeddings)

```

```

Phase3 > phase3.py > ...
123
124     # We will search for all candidate documents to get their cosine similarity scores with the query,
125     # and then combine these scores with the normalized BM25 scores using a weighted sum to get a final
126     # score for each candidate document. Finally, we will sort the candidates based on this final
127     num_candidates = len(candidate_texts)
128     semantic_distances, indices = index.search(query_embeddings, num_candidates)
129
130     semantic_scores_map = {}
131     for local_idx, dist in zip(indices[0], semantic_distances[0]):
132         semantic_scores_map[local_idx] = dist
133
134     # We will use a weight w to balance the contribution of BM25 and semantic similarity scores.
135     w = 0.7 #BM25 had a better performance than semantic similarity in phase1 and phase2, so we give it a higher weight.
136     weighted_results = []
137
138     for i in range(num_candidates):
139         doc_id = candidate_ids[i]
140         s_bm25 = norm_bm25[i]
141         s_semantic = semantic_scores_map.get(i, 0.0)
142
143         final_score = (w * s_bm25) + ((1 - w) * s_semantic)
144         weighted_results.append((doc_id, final_score))
145
146     weighted_results.sort(key=lambda x: x[1], reverse=True)
147

```

Έτσι, καταφέραμε να έχουμε τα αποτελέσματα που φαίνονται παρακάτω και είναι σαφώς καλύτερα από αυτά των μη υβριδικών μοντέλων των πρώτων 2 φάσεων της εργασίας.

```

Phase3 > eval3_20.txt
1    map                all 0.6824
2    P_5                all 0.9000
3    P_10               all 0.7700
4    P_15               all 0.7000
5    P_20               all 0.6050

```

```

Phase3 > eval3_50.txt
1    map                all 0.7752
2    P_5                all 0.9000
3    P_10               all 0.7700
4    P_15               all 0.7000
5    P_20               all 0.6050

```

```

Phase3 > eval3_30.txt
1    map                all 0.7493
2    P_5                all 0.9000
3    P_10               all 0.7700
4    P_15               all 0.7000
5    P_20               all 0.6050

```

Φάση 1:

	k=20	k=30	k=50
MAP	0.6895	0.7471	0.7701
P@5	0.8800	0.8800	0.8800
P@10	0.7300	0.7300	0.7300
P@15	0.7000	0.7000	0.7000
P@20	0.6300	0.6300	0.6300

Φάση 2:

	k=20	k=30	k=50
MAP	0.3121	0.3413	0.3744
P@5	0.6600	0.6600	0.6600
P@10	0.5000	0.5000	0.5000
P@15	0.3933	0.3933	0.3933
P@20	0.3500	0.3500	0.3500

Φάση 3:

	k=20	k=30	k=50
MAP	0.6824	0.7493	0.7752
P@5	0.9000	0.9000	0.9000
P@10	0.7700	0.7700	0.7700
P@15	0.7000	0.7000	0.7000
P@20	0.6050	0.6050	0.6050

Η Elasticsearch φιλτράρει τα έγγραφα που δεν έχουν καμία κοινή λέξη, ενώ ο Transformer καταλαβαίνει ποιο από τα υποψήφια έγγραφα απαντάει πραγματικά στο νόημα της ερώτησης. Η υβριδική προσέγγιση απέδειξε την υπεροχή της στην ακρίβεια των πρώτων αποτελεσμάτων. Η βελτίωση στα P@5 και P@10 σε σχέση με το BM25 αποδεικνύει ότι ο συνδυασμός λεκτικής και σημασιολογικής πληροφορίας βοηθά στον εντοπισμό των πιο σχετικών εγγράφων. Η μικρή κάμψη στο MAP στα k=20 και στο P@20 αποδίδεται στο γεγονός ότι ο Transformer, στην προσπάθειά του να αξιολογήσει το νόημα, ενδέχεται να υποβαθμίσει έγγραφα που περιέχουν κρίσιμους όρους (keywords) αλλά έχουν χαμηλότερη διανυσματική συνάφεια.

Η υβριδική προσέγγιση συνδυάζει τα πλεονεκτήματα και των δύο μοντέλων: την ταχύτητα και την ακρίβεια όρων του BM25 με τη βαθιά κατανόηση γλώσσας των Transformers. Έτσι, ο Transformer έχει πλέον τη δύναμη να επηρεάσει την τελική σειρά και να "ανεβάσει" έγγραφα που είναι σημασιολογικά σχετικά, ακόμα κι αν η Elasticsearch δεν τους έδωσε το υψηλότερο σκορ.

Τα αρχεία του φακέλου Phase3 είναι:

[documents.csv](#) (μας παρέχεται και περιλαμβάνει τα κείμενα)

[queries.csv](#) (μας παρέχεται και περιλαμβάνει τα ερωτήματα)

[qrels.txt](#) (μας παρέχεται και περιλαμβάνει τις σωστές απαντήσεις)

[trec_eval.exe](#) (το εργαλείο αξιολόγησης)

[cyqwin1.dll](#) (απαραίτητο αρχείο του εργαλείου)

[phase3.py](#) (το πρόγραμμα μας όπου μπορούμε να το τρέξουμε και να πάρουμε τα παρακάτω αρχεία)

[documents.jsonl](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα επεξεργασμένα κείμενα που χρησιμοποιούνται στο BM25)

[results3_20](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 20 πρώτα ανακτηθέντα κείμενα)

[results3_30](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 30 πρώτα ανακτηθέντα κείμενα)

[results3_50](#) (το παράγει το πρόγραμμα και περιλαμβάνει τις απαντήσεις για τα 50 πρώτα ανακτηθέντα κείμενα)

[eval3_20](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 20 πρώτα ανακτηθέντα κείμενα)

[eval3_30](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 30 πρώτα ανακτηθέντα κείμενα)

[eval3_50](#) (το παράγει το πρόγραμμα και περιλαμβάνει τα αποτελέσματα της αξιολόγησης για τα 50 πρώτα ανακτηθέντα κείμενα)

Η εργασία πραγματοποιήθηκε κατά το χειμερινό εξάμηνο του έτους 2025-2026 στα πλαίσια του μαθήματος “Συστήματα Ανάκτησης Πληροφοριών” από τις φοιτήτριες του τμήματος Πληροφορικής στο Οικονομικό Πανεπιστήμιο Αθηνών, [Στυλιανή Μουμτζή \(3220127\)](#) και [Αναΐς Φαρχάτ \(3220280\)](#).